Day 2 – Core Models: Publisher, Game, GameKey

🏷️ Setup & Prerequisites

1. **Setup Checklist**:
   - **Virtual Environment Active**: Confirm that your terminal has your virtual environment active and shows `(.venv)` in the prompt. If not, run `source .venv/bin/activate` (or `.venv\Scripts\activate` on Windows).
   - **IDE Ready**: Open the project folder in your IDE, specifically opening `games/models.py` and `games/admin.py`.

2. **Prerequisites**:
   - You must have a scaffolded Django project (completed from Day 1) with the development server verified and running.

🎯 Learning Objectives

By the end of this class, you will be able to:

- Explain Django's ORM (Object-Relational Mapper) mental model.
- Define database relationships using `ForeignKey` and `OneToOneField`.
- Apply cascading rules on relationships using `on_delete=models.CASCADE`.
- Generate and run database migrations using `makemigrations` and `migrate`.
- Register models with Django's administrative interface and manage data through the admin UI.

⏱️ Classroom Timeline (45 min)

Segment	Duration	Focus
1. Hook & Big Picture	10 min	Discuss ORM mental models, design the entities, and draw relationship diagrams.
2. Key Concepts & Core Ideas	7 min	Define fields, relationship types, cascade deletion, and migrations.

3. Live Coding Walkthrough	20 min	Live-code the models, run database migrations, and configure administrative site.
4. Check for Understanding	5 min	Q&A on relationship choices, migrations, and model properties.
5. Class Wrap-up & Checkpoint	3 min	Verify models are editable in the admin UI.

🚀 Lecture Step-by-Step Delivery Plan

1. Hook & Big Picture (10 min)

Before writing code or routing APIs, we must build a strong foundation: the database schema.

- **ORM Mental Model**: Django models are Python classes. Class attributes represent database columns, and instances of these classes represent database rows.
- **Relationships**: Our database design links these entities:
  - **Publisher** has a one-to-one link to Django's built-in **User** (for logging in).
  - **Game** belongs to a single **Publisher** (foreign key, one-to-many).
  - **GameKey** belongs to a single **Game** (foreign key, one-to-many), and optionally has a **User** owner (when purchased).

2. Key Concepts & Core Ideas (7 min)

Introduce these essential terms:

- **ORM (Object-Relational Mapper)**: Bridges the gap between OOP in Python and Relational SQL databases (SQLite for development, PostgreSQL for production). You work entirely with Python objects, and Django handles writing SQL queries like `SELECT`, `INSERT`, `UPDATE`, and `DELETE`.
- **`ForeignKey` VS `OneToOneField`**:
  - `ForeignKey` establishes a one-to-many relationship (one publisher can release many games).
  - `OneToOneField` enforces uniqueness (one user can represent only one publisher).
- **`on_delete=models.CASCADE`**: When a parent object is deleted, all child objects referencing it are deleted.
  - **Question**: What should happen to GameKeys if we delete a Game?
    - *Answer*: Since keys depend on a game's existence, deleting the game cascades the deletion, deleting all associated GameKeys. Contrast this with `SET_NULL` (retaining keys with null game fields) or `PROTECT` (blocking game deletion while keys exist).
- **`null=True` VS `blank=True`**: `null` is database-level (allows database cells to be NULL), while `blank` is validation-level (allows empty inputs in forms/serializers).

- **Migrations**: A version-control system for the database schema.
  - **Question**: Why do we say migrations are versioned schema changes, not magic?
    - *Answer*: Migrations are Python files representing a history of changes. Each script defines the forward schema change and the reverse rollback step, allowing Git to track database schema evolution over time.

---

3. Live Coding Walkthrough (20 min)

Step 3.1: Defining the Models

- **Concept**: Open `games/models.py`. We will define the three core models. `GameKey.owner` uses `null=True, blank=True` because keys exist before being sold and having an owner.
- **Code**: Modify `games/models.py`:

```
from django.db import models
from django.contrib.auth.models import User


class Publisher(models.Model):
    # CharField is used for short text strings. max_length specifies the maximum characters
    name = models.CharField(max_length=200)

    # URLField validates that the input is a valid URL string.
    webhook_url = models.URLField()

    # Stores the secret token used to sign webhook requests for security.
    webhook_secret = models.CharField(max_length=64)

    # OneToOneField creates a 1-to-1 relationship. Each publisher has exactly one Django Use
    # on_delete=models.CASCADE ensures that if the Django User is deleted, the Publisher pro
    user = models.OneToOneField(User, on_delete=models.CASCADE)

    # The __str__ method defines the string representation of the model instance.
    # This is what appears in the Django Admin panel dropdowns and listings.
    def __str__(self):
        return self.name


class Game(models.Model):
    title = models.CharField(max_length=200)

    # ForeignKey creates a one-to-many relationship: a publisher can have many games, but a
    publisher = models.ForeignKey(Publisher, on_delete=models.CASCADE)

    # DecimalField is used for storing currency to prevent floating-point rounding issues.
    # max_digits=6 allows values up to 9999.99, and decimal_places=2 specifies two decimal p
    price = models.DecimalField(max_digits=6, decimal_places=2)

    def __str__(self):
        return self.title
```

```
class GameKey(models.Model):
    # Choices define a static list of options for the status field.
    STATUS_CHOICES = [('active', 'Active'), ('expired', 'Expired')]

    # unique=True guarantees that no two game keys can share the exact same string.
    key_string = models.CharField(max_length=50, unique=True)
    game = models.ForeignKey(Game, on_delete=models.CASCADE)

    # choices limits inputs to values in STATUS_CHOICES. The default is 'active'.
    status = models.CharField(max_length=10, choices=STATUS_CHOICES, default='active')

    # DateTimeField stores date and timezone-aware time stamps.
    expires_at = models.DateTimeField()

    # owner is optional. Before purchase, owner is null. Thus null=True and blank=True are req
    owner = models.ForeignKey(User, on_delete=models.CASCADE, null=True, blank=True)

    def __str__(self):
        return self.key_string
```

- **Verify**: Run `python manage.py check` to ensure there are no syntax or configuration errors in the models.
- ⚠ **Common Pitfalls I will flag**:
  - **Missing `__str__` method**: If you omit `__str__`, the admin UI defaults to unhelpful strings like "Publisher object (1)".
  - **Confusing `null` and `blank`**: If you write `blank=True` but forget `null=True` on nullable database fields, Django will throw database integrity errors (`NOT NULL constraint failed`) when saving empty records.

Step 3.2: Database Migrations

- **Concept**: Django scans models for modifications and creates schema migration instructions, then executes them against the database.
- **Code**:

```
# Step 1: Scan models.py for modifications and generate a blueprint file
python manage.py makemigrations

# Step 2: Apply the blueprint to the active SQLite/PostgreSQL database to create/update tables
python manage.py migrate
```

- **Verify**: Open the generated migration file (e.g., `games/migrations/0001_initial.py`) and read its contents to understand how Django translates models into schemas.
- ⚠ **Common Pitfalls I will flag**:
  - **Forgetting `'games'` app in `INSTALLED_APPS`**: If the app is missing from `settings.py`,

```
will report "No changes detected".
```

- ○ **Running `migrate` before `makemigrations`**: Nothing will happen. Remember the two-step migration lifecycle: compile blueprint first (`makemigrations`), apply next (`migrate`).

Step 3.3: Registering Models in Django Admin

- **Concept**: To inspect and manage our models, we will register them with Django's built-in administration panel.
- **Code**: Modify `games/admin.py`:

```
from django.contrib import admin
from .models import Publisher, Game, GameKey

# Registering the models exposes them in the administrative UI dashboard
admin.site.register(Publisher)
admin.site.register(Game)
admin.site.register(GameKey)
```

- **Verify**: Create a superuser and log into the admin panel to test model registration:

```
# Create admin credentials in the terminal
python manage.py createsuperuser
```

Run the server (`python manage.py runserver`), navigate to `http://127.0.0.1:8000/admin`, log in, and ensure you can create/edit a `Publisher`, a `Game`, and a `GameKey` manually.

---

4. Check for Understanding (5 min)

> ***Question***: *"Why does `GameKey.owner` use `null=True, blank=True` but `GameKey.game` does not?"*
> ***Answer***: *`GameKey.owner` is optional because a game key can exist (e.g., pre-loaded by a publisher) before any user purchases it. Thus, the database field must allow `NULL` (`null=True`) and API/form validation must allow it to be empty (`blank=True`). Conversely, a `GameKey` must always belong to a specific game, so `GameKey.game` is mandatory and cannot be null.*

> ***Question***: *"What's the difference between `makemigrations` and `migrate`? What does each file represent?"*
> ***Answer***: *`makemigrations` scans model definitions for changes and creates new, numbered migration files (e.g., `0001_initial.py`), which are blueprints describing how to modify the database schema. `migrate` actually executes those blueprints against the database, applying the changes to create/modify tables. The migration files represent versioned history of the schema and must be committed to git.*

> ***Question**: "If we delete a `Publisher`, what happens to their `Game` records? What about their `GameKey` records?"*
> ***Answer**: Because `Game.publisher` uses `on_delete=models.CASCADE`, deleting a `Publisher` automatically cascades and deletes all related `Game` records. In turn, because `GameKey.game` also uses `on_delete=models.CASCADE`, deleting those `Game` records will automatically cascade and delete all associated `GameKey` records.*

> ***Question**: "Why is `key_string` marked `unique=True`?"*
> ***Answer**: A game key represents a single, unique digital asset. If it were not marked `unique=True`, the database could accidentally store duplicate key strings, leading to the same key being issued to multiple users, violating licensing rules and causing validation collisions.*

---

5. Daily Checkpoint & Wrap-up (3 min)

- **Verification Criteria**:
  - ○ `python manage.py migrate` completes with no errors.
  - ○ You are logged into the `/admin` interface.
  - ○ You can create a `Publisher` record, a `Game` record, and a `GameKey` record successfully using the admin UI.